AI Assisted coding

B.Devendar

2303A53036

Batch-46

Task 1 – Input Validation Check

Objective: Analyze an AI-generated Python login script for input validation vulnerabilities.

- **AI Prompt used:** *"Generate a simple Python username and password login program."*
- **Vulnerability Identified:** The initial code accepted any string length and special characters, leaving the system open to potential injection or buffer overflow issues.
- **Secure Refactored Code:**

Python

```
import re


def validate_login(username, password):
    # Enforcing 4-12 alphanumeric characters for username
    user_regex = r"^[a-zA-Z0-0]{4,12}$"
    if re.match(user_regex, username) and len(password) >= 8:
        return True
    return False


user = input("Username: ").strip()
pw = input("Password: ").strip()


if validate_login(user, pw):
    print("Input format secured.")
```

```
else:
    print("Invalid format. Use 4-12 alphanumeric characters.")
```

Task 2 – SQL Injection Prevention

Objective: Refactor database queries to use parameterized statements.

- **AI Prompt used:** *"Ask AI to generate a Python script using SQLite to fetch user details."*
- **Vulnerability Identified:** Use of f-strings (e.g., f"SELECT * FROM users WHERE name = '{name}'") allows SQL Injection.
- **Secure Refactored Code:**

Python

```
import sqlite3


def get_user_secure(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    # Using Parameterized Queries (Prepared Statements)
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))
    return cursor.fetchone()
```

Task 3 – Cross-Site Scripting (XSS) Check

Objective: Prevent malicious scripts from executing in a web form.

- **AI Prompt used:** *"Generate a feedback form with JavaScript-based output."*
- **Vulnerability Identified:** Direct rendering of user input using .innerHTML.
- **Secure Refactored Code:**

JavaScript

```
function displayFeedback() {
    const input = document.getElementById('user-input').value;
    const display = document.getElementById('display');
    const p = document.createElement('p');
    // .textContent automatically escapes HTML entities
    p.textContent = input;
    display.appendChild(p);
}
```

Task 4 – Security Audit (Side-by-Side)

Scenario: API Integration with Hardcoded Credentials.

Feature	**Insecure Version**	**Secure Version**
Credential Handling	api_key = "12345-ABCDE"	api_key = os.getenv("API_KEY")
Risk	Hardcoded secrets leak in version control.	Secrets are stored in the environment.
Audit Result	Vulnerable to credential theft.	Compliant with security standards.

Task 5 – Insecure Data Serialization Check

Objective: Replace pickle with safer JSON formats.

- **AI Prompt used:** *"Generate a script using pickle.load()."*
- **Why Pickle is Dangerous:** pickle can execute arbitrary code during the loading process, leading to Remote Code Execution (RCE).
- **Secure Refactored Code:**

Python

```
import json
```

```
# Using JSON is safe as it only handles data, not executable objects
def load_data_securely(filename):
    with open(filename, 'r') as f:
        return json.load(f)
```